

OS_HW2

MULTI-THREADED WORD COUNT

모바일 시스템 공학과

32201345

나웅철

목차 :

- 스레드의 개념
- 멀티스레드 관련 개념
- 동기화란?
- 효율적인 동기화란? with 동기화 빈도, 임계구역 범위 그리고 성능
- 생산자-소비자 _ 가장 빠른 실행을 목표로..
- 주요 함수 소개
- 구현 계획
- Version 1 구현 및 문제점 (작업량 1줄, 통계 저장공간 공유) _ r8 ver.
- Version 2 구현 및 한계점 (작업량 1000줄 이상, 통계 저장공간 공유) _ r6 ver.
- Version 3 구현 및 한계점 (작업량 1줄, 통계 저장공간 분리) _ r11 ver.
- Version 4 구현 및 한계점 (작업량 최대 천만 줄 , 통계 저장공간 분리) _ r5 ver.
- 마치며..

스레드

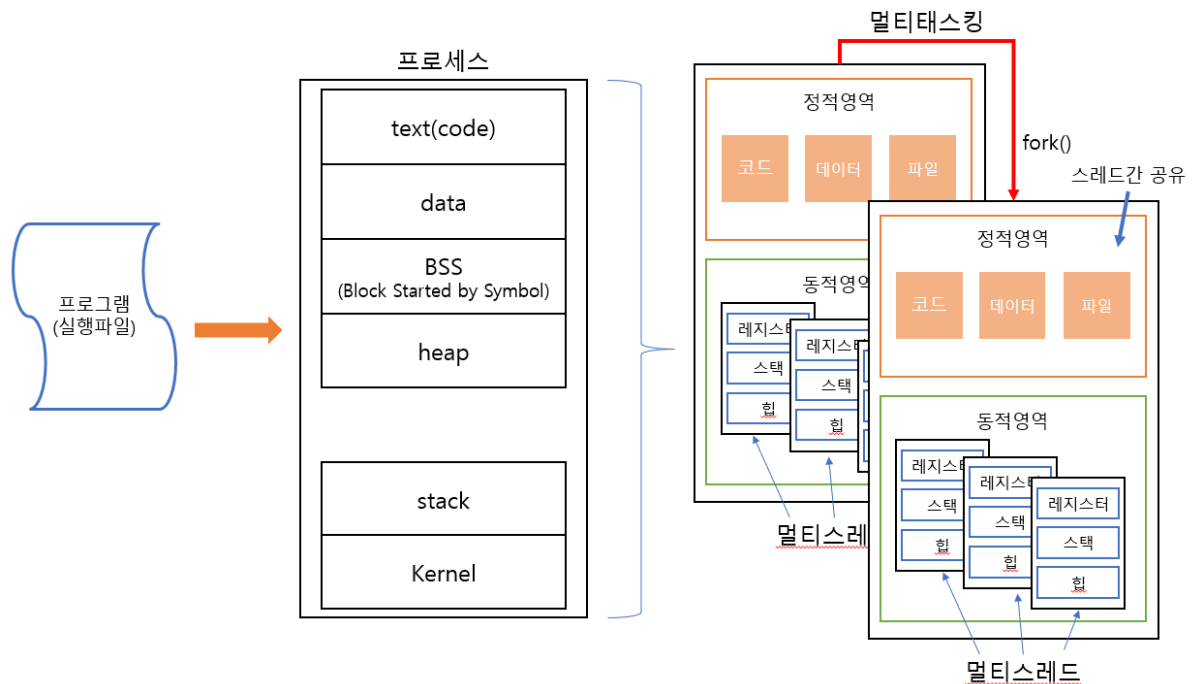
프로세스는 실행파일 형태의 프로그램이 운영체제에 의해 다뤄질 수 있는 형태의 인스턴스라고 한다면, 스레드는 그러한 프로세스 내에서 실행되는 가장 작은 흐름의 단위라고 할 수 있다. 하나의 프로세스는 하나 이상의 스레드를 가질 수 있다. 각 스레드는 독립적으로 실행되지만 모두 하나의 프로세스내의 흐름이기 때문에 하나의 프로세스에 속한 모든 스레드들은 해당 프로세스의 자원을 공유한다.

구분	프로세스	스레드
주소 공간	독립적인 메모리 공간을 가짐	같은 프로세스 내에서 주소 공간을 공유
자원 공유	각 프로세스는 자원을 독립적으로 가짐	프로세스 내에서 자원을 공유
오버 헤드	생성 및 전환 시 오버헤드가 큼	생성 및 전환 시 오버헤드가 적음
안정성	다른 프로세스의 오류가 영향을 미치지 않음	한 스레드의 오류가 전체 프로세스에 영향을 줄 수 있음
커뮤니케이션	IPC와 같은 별도의 통신 방식 필요	직접 메모리를 공유하므로 빠르고 간편하게 통신 가능

스레드의 특징을 좀 더 자세히 기술하자면, 다음과 같다. 1)스레드 프로세스의 메모리(주소 공간), 전역 변수, 파일 디스크립터 등의 자원을 공유한다. 이를 이용하여 스레드들 간의 데이터 교환이 일어난다. 2)각 스레드는 자신의 스택(stack), 프로그램 카운터(program counter), 레지스터를 가지고 있어 독립적으로 실행된다. 자신만의 ~을 갖는다는 말은 곧 어느 한 스레드가 중단되더라도 다른 스레드들은 계속 실행될 수 있다는 의미이다.

스레드의 장점 및 단점은 다음과 같다.

	장점	단점
1.	효율적으로 멀티태스킹이 가능	동기화 문제
2.	메모리 사용량이 적고 오버헤드가 낮음	디버깅 난이도
3.	스레드들간 메모리를 통한 데이터 교환	한 스레드의 문제가 프로세스의 문제로 확산



[그림 1] 프로그램-스레드

멀티스레드 관련 개념

Key word

- 경쟁 상태(Race Condition) : 추후 기술
- 상호 배제(Mutual Exclusion) : 한 번에 하나의 스레드만 특정 자원에 접근할 수 있도록 보장하는 것
- 임계 구역(Critical Section) : 공유 자원에 접근하는 코드 블록 // 경쟁 상태가 발생할 수 있는 코드 부분

멀티스레딩(Multithreading)은 [그림1]에서 보듯 하나의 프로세스 내에서 해당 프로세스의 자원을 이용하는 여러 흐름들이 동시에 병렬적으로 수행되는 방식을 말한다. 이러한 방식을 이용하면 CPU 코어를 효율적으로 활용할 수 있고 응답 시간을 줄일 수 있다. **하지만 꼭 하나의 프로세스 내에서 여러 스레드로 병렬처리 하는 것이 성능 향상과 직결되는 것은 아니다.** 여러 스레드가 동시에 실행되면서 같은 자원에 접근할 때 데이터 불일치 문제가 발생할 수 있고, 해당 영역에 접근하기위해 기다리느라 오버헤드가 커질 수도 있기 때문이다. 따라서 멀티스레딩이 성능의 향상으로 이루어지기 위해서는 해당 프로그램에 맞는 최적화된 동기화 기법을 사용해야 한다. 또한 다른 스레드들이 기다리는 시간을 줄이기 위해 임계 구역을 꼭 필요한 부분으로 한정 지어 설정해야 한다. 마지막으로 동기화가 필요한 상황이 적게 발생하도록 설계하는 것도 방법이 될 수 있

다.

동기화란?

동기화(Synchronization)란, 멀티스레딩이나 멀티프로세싱 환경에서 여러 스레드나 프로세스가 공유 자원에 안전하게 접근할 수 있도록 하는 기법을 말한다. 즉, 동기화는 데이터의 무결성과 일관성을 보장하기 위해, 여러 작업이 동시에 공유 자원을 사용할 때 자원의 접근 순서를 조정하는 것을 의미한다. 여러 작업이 동시에 공유 자원을 사용하는 것이 문제가 되는 이유는 구조적으로 해당 데이터를 이용해 작업을 하고 나면 그 값을 최신화 하게 되는데, 먼저 접근한 작업의 결과가 최신화되기 전에 그 자원을 이용하게 되면 이전 작업이 무효가 되기 때문이다.

주요 동기화 기법: 뮤텝스, 세마포어, 스핀락

뮤텝스(Mutual Exclusion)는 특정 자원에 동시에 접근하는 스레드의 수를 하나로 제한하여, 한번에 하나의 스레드만 자원에 접근하도록 보장하는 방식이다. 특정 자원에 대해 잠금(lock)하고 사용이 끝나면 다시 잠금을 해제(unlock)하는 방식으로 동작하며, 이를 통해 상호 배제를 구현한다.

```
while(1){
    pthread_mutex_lock(&sp->lock);
    //lock을 얻은 경우에만 전역변수 count를 조작 가능

    //임계구역//
    count ++; //공유자원

    //작업 끝, 잠금 해제
    pthread_mutex_unlock(&sp->lock);
}
```

그림 2 뮤텝스

세마포어(Semaphore)는 자원 접근을 제한하는 카운팅 방식의 동기화 메커니즘이다. 세마포어는 특정 자원에 여러 스레드가 접근할 수 있도록 허용하며, 카운트를 통해 동시 접근 가능한 스레드의 수를 관리한다.

```

//세마포어 초기화
sem_init(&sp->full,0,0);
sem_init(&sp->empty,0,100);

//프로듀서 스레드의 경우 빈 공간에 데이터를 저장하는 역할이므로..
sem_wait(&sp->empty); //empty가 0이 아닐때까지 기다렸다가 1을 빼면서 진입
// 임계 구역
sem_post(&sp->full); //full을 1 더하고 진입 종료

//컨슈머 스레드의 경우 데이터가 있는 공간으로부터 읽어오는 역할이므로..
sem_wait(&sp->full); //full이 0이 아닐때까지 기다렸다가 1을 빼면서 진입
// 임계 구역
sem_post(&sp->empty); //empty를 1 더하고 진입 종료

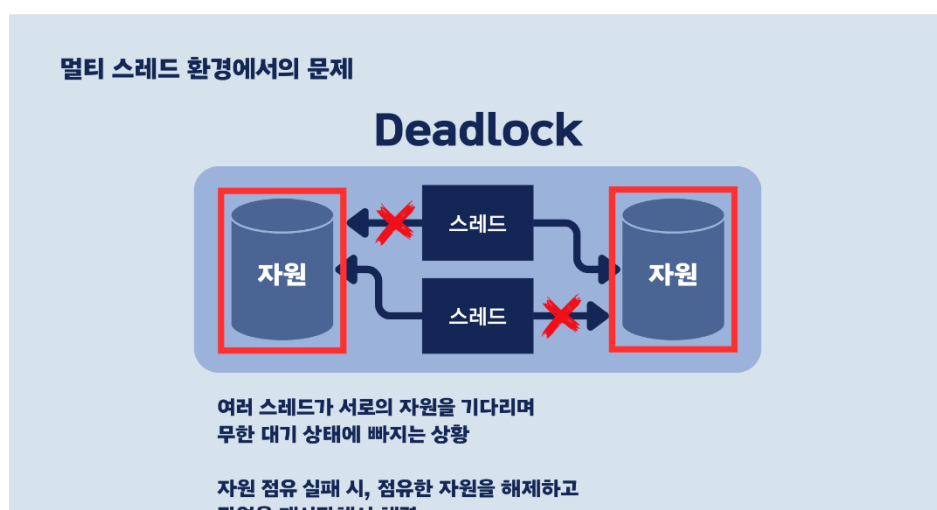
```

그림 3 세마포어

스핀락(Spinlock)은 자원을 사용할 수 있을 때까지 계속해서 자원을 점유하려는 시도를 반복하는 방식으로, 바쁜 대기(Busy Waiting)를 통해 락을 획득하는 방식이다. 짧은 대기 시간 안에 락이 해제될 가능성이 높다면 무방하지만, 대기 시간이 길어질 경우 반복되는 시도 때문에 CPU자원이 낭비될 수 있다는 단점이 있다.

동기화와 관련된 문제: 데드락(Deadlock), 기아 상태(Starvation), 경쟁 상태(Race Condition)

데드락은 두 개 이상의 스레드가 서로 자원을 점유한 상태에서 서로의 자원이 해제되기를 기다리며 영원히 멈춰 있는 상태를 의미한다. 상호 배제와 순환 대기가 있을 때 발생 할 수 있으며, 프로그램의 정지상태를 초래한다.



[그림 4] 데드락

기아 상태는 특정 스레드가 자원에 접근할 기회를 얻지 못하고 무한정 대기하는 상황을 말한다. 우선순위가 낮은 스레드가 높은 우선순위의 스레드에게 계속 밀리거나, 락이 자주 점유되어 특정 스레드가 기아 상태에 빠질 수 있다.

```
int flag[2] = {false, false}; // 두 스레드의 진입 의사 표시
int turn = 0;                // 누구의 차례인지 나타냄

// 프로세스 0
flag[0] = true;               // 프로세스 0이 진입 의사 표시
turn = 1;                    // 상대 프로세스에게 차례를 넘김
while (flag[1] && turn == 1); // 프로세스 1이 진입하려고 하면 대기

// Critical Section (임계 구역)
```

[그림 5] turn 변수를 이용한 기아 상태 방지 _ 피터슨 알고리즘 중

경쟁 상태는 여러 스레드가 동시에 같은 자원에 접근하면서 예상치 못한 데이터 불일치가 발생하는 문제를 의미한다. 경쟁 상태는 동기화 메커니즘이 적절하게 적용되지 않을 때 발생할 수 있으며, 예측할 수 없는 결과를 초래한다.

효율적인 동기화란? With 동기화 빈도, 임계구역, 성능

하나의 프로세스 내에서 멀티스레딩, 즉 굳이 작업을 분할하고 여러 스레드에게 분배하여 병렬 처리하도록 하는 이유는 분명히 작업을 효율적이게 하기 위함이라는 목적이 있다. 하지만 앞서 살펴봤듯 그저 스레드를 여러 개 사용한다고 해서는 각종 동기화와 관련된 문제들이 발생하기 십상이기 때문에 오히려 효율적이지 못한 결과가 나올 확률이 높아진다. 그렇다면 무엇이 효율적인 동기화일까? 멀티스레딩 작업에서 효율적인 동기화란, 여러 스레드가 동시에 실행되면서 **최소한의 동기화로 최대한의 병렬성을 유지하여** CPU 자원과 시스템의 성능을 최적화하는 것을 의미한다. 효율적인 동기화는 자원의 충돌을 방지하고, **동시에 불필요한 대기 시간과 자원 낭비를 최소화**하는 것을 목표로 한다. 즉 정리하면 다음과 같다.

“최소한의 동기화, 최소한의 대기시간, 최소한의 자원 낭비”

여러 스레드들을 사용하려면 분명 동기화가 필요한 것은 사실이다. 하지만 추후 실습결과에서 살펴보겠지만 동기화 자체가 성능에 도움이 되는 것은 아니다. 오히려 동기화의 높은 빈도는 성능 저하의 원인이 된다. 심지어는 차라리 단일 스레드를 이용하는 것이 훨씬 빠른 결과를 도출하기도 한다. 정확히 말하면 과도한 상호 배제가 원인인데, 단일 자원에 대해 과도한 상호 배제를 하면서 작업을 수행할 경우 다른 여러 스레드들이 해당 자원에 대한 접근 권한을 얻기 위해 기다리

게 되면서 병렬성을 저해할 수 있기 때문이다. 이러한 관점에서 봤을 때 임계 구역의 긴 범위 설정도 비슷한 이유로 오버헤드를 발생시킬 수 있다. 굳이 임계구역에 포함시키지 않아도 될 부분들, 즉 여러 스레드가 동시에 작업을 진행해도 서로에게 영향을 주지 않는 부분들 까지도 임계구역에 포함시키지 않도록 주의해야 한다. 임계구역이 짧으면 짧을수록 스레드가 할 일만 하고 빠르게 다음 스레드에게 작업 권한을 넘겨줄 수 있으므로 다른 스레드들이 해당 자원을 기다리는 시간이 줄어들기 때문이다. 지금까지 살펴본 2가지 방법들(최소한의 상호 배제, 짧은 임계 구역)은 스레드들의 대기시간을 최소화하는 것에 초점을 맞춘 방법들이었는데 아예 대기자체를 하지 않도록 경쟁 상태 자체를 최소화하는 방법도 있다. 물론 이 방법은 공유 자원 사용을 줄여 경쟁상태를 최소화하므로 자원을 공유하지 않는 만큼 추가로 메모리 소모는 하겠지만 그 소모량이 작을 경우 충분히 효율적인 방법이라고 판단할 수 있다.

생산자-소비자 – 가장 빠른 실행을 목표로..

이번에 주어진 과제는 Producer-Consumer 컨셉의 멀티스레딩 프로그램을 구현하고 동기화를 최대한 활용하여 가장 빠른 실행을 도출해내는 것이었는데, 필자는 이를 최대한 효율적인 병렬처리를 통한 빠른 결과 도출을 의미하는 것으로 이해하였다. 사실 처음에는 동기화를 최대한 활용한다는 것을 잘못 이해하여 상호배제를 최대한 많이 사용해야 하는 것으로 생각하여 "도대체 왜 스레드 수를 늘렸는데 오히려 실행 시간이 더 오래 걸릴까?"하는 고민에 상당히 많은 시간을 할애하였고 그러한 근본적인 관점 자체가 틀렸음을 깨닫게 되었다. 단순히 스레드만 여러 개 생성하고 동기화 도구를 사용한다고 되는 것이 아니라 병렬적으로 잘 작동할 수 있게끔 최적화를 해야 했던 것이다.

생산자-소비자 예제의 기본적인 구조는 다음과 같다.

실행 구조

- 사용자로부터 명령줄 인수로 텍스트파일과 생산자 스레드 개수, 소비자 스레드 개수를 받는다
- 각 스레드를 생성하고 생성한 스레드들은 각기 할당된 함수들을 독립적으로 실행한다
 - 생산자 스레드는 텍스트 파일로부터 데이터를 읽어와서 공유 자원에 저장한다
 - 소비자 스레드는 공유자원으로부터 데이터를 가져와서 통계를 낸다
- 통계를 출력한다

```

//메인 함수 흐름
int main(int argc, char* argv[]){
    FILE* rfile;

    if(argc == 1){ // 명령줄 인수가 1개인 경우 -> exit
        printf("usage: ./prod_cons <readfile> #Producer #Consumer\n");
        exit (0);
    }

    rfile = fopen((char*)argv[1],"rb");
    if(rfile == NULL){//읽어올 텍스트 파일이 NULL인 경우 -> exit
        printf("file error");
        exit(0);
    }

    if(argv[2] != NULL){//프로듀서 스레드 개수 결정

    }

    if(argv[3] != NULL){//컨슈머 스레드 개수 결정

    }

    for(){ //프로듀서 스레드 생성

    }

    for(){ // 컨슈머 스레드 생성

    }

    for(){ //프로듀서 스레드 종료 대기

    }

    for(){ //컨슈머 스레드 종료 대기

    }

    statistics(sp); // 통계 함수 호출
    pthread_exit(NULL); //메인 스레드 종료
    exit(0); // 모든 스레드 종료를 보장하면서 프로그램 종료
}

```

[그림 6] 메인 함수 흐름

주요 라이브러리 소개

이번 과제를 수행하면서 난생처음 보는 라이브러리들과 함수, 변수 타입들을 사용해보았는데 일반적으로 학생의 입장에서 접하기 힘든 라이브러리기 때문에 유의미하다고 생각하여 짧게 보고서 내용에 포함하고자 한다.

```
#include <pthread.h>
#include <semaphore.h>
```

[그림 7] 주요 라이브러리

```
//스레드 생성, 생성한 스레드가 수행할 함수 지정
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg);

//해당 스레드가 종료할 때 까지 메인 스레드 대기
int pthread_join(pthread_t thread, void **retval);

//뮤텍스 락/언락
pthread_mutex_lock() / pthread_mutex_unlock()

//조건 변수 사용
pthread_cond_wait() / pthread_cond_signal()

//스레드 종료
void pthread_exit(void *retval)

//스레드를 나타내는 데이터 타입(스레드 식별자로 사용)
pthread_t

//뮤텍스를 나타내는 데이터 타입
pthread_mutex_t

//세마포어를 나타내는 데이터 타입
sem_t
```

[그림 8] pthread 라이브러리

```
//세마포어 초기화
int sem_init(sem_t *sem, int pshared, unsigned int value);

//해당 세마포어 카운트가 0 이상일 때 까지 대기하다 1감소 시키고 임계구역 진입
sem_wait();
//해당 세마포어 카운트를 1 증가시키고 임계구역 진입 종료
sem_post();
```

[그림 9] semaphore 라이브러리

구현 계획

버전 별 컨셉

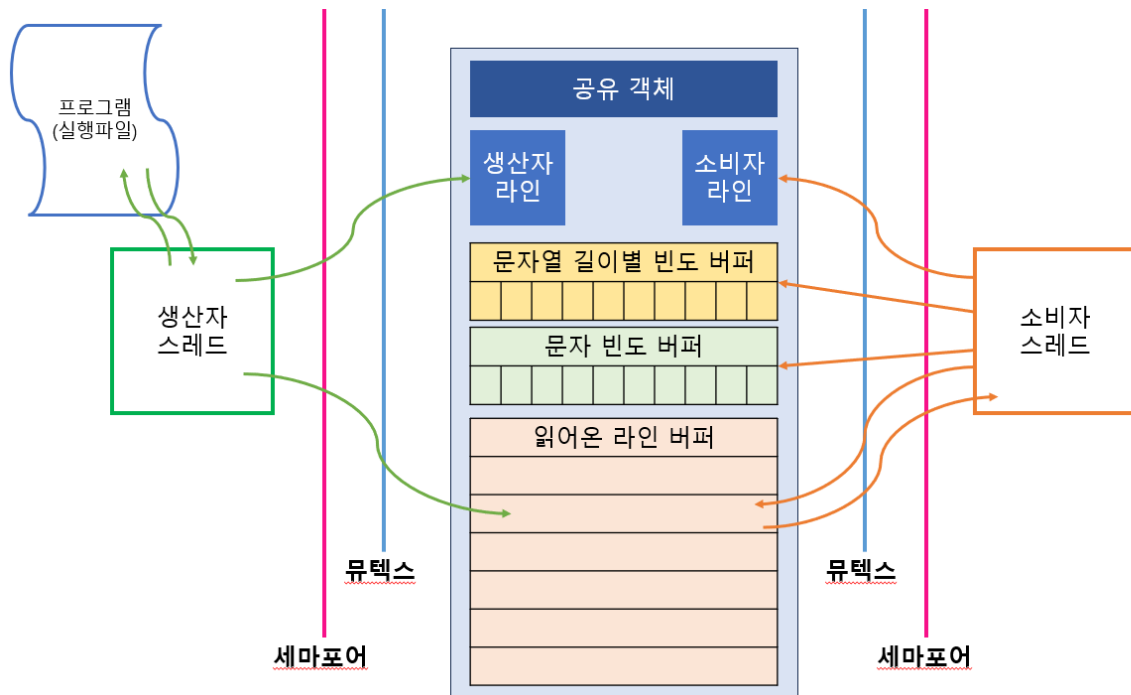
- Ver.1 : 기본형
 - 생산자 스레드 : 생산자 라인 변수 공유(파일로부터 읽어올 줄, 저장할 버퍼번호)
 - 소비자 스레드 : 소비자 라인 변수 공유(읽어올 버퍼 번호), 통계 버퍼 2가지 공유
 - 작업 단위 : 텍스트 파일 한 줄

- Ver 2 : 기본형에서 작업단위를 늘린 버전
 - 생산자 스레드 : 생산자 라인 변수 공유(파일로부터 읽어올 줄, 저장할 버퍼번호)
 - 소비자 스레드 : 소비자 라인 변수 공유(읽어올 버퍼 번호), 통계 버퍼 2가지 공유
 - 작업 단위 : 텍스트 파일 여러 줄(1000줄, 10000줄..)

- Ver 3 : 기본형에서 상호 배제를 줄인 버전
 - 생산자 스레드 : 생산자 라인 변수 공유(파일로부터 읽어올 줄, 저장할 버퍼번호)
 - 소비자 스레드 :
 - ① 소비자 라인 변수 공유(읽어올 버퍼 번호)
 - ② 통계 결과 저장할 독립된 버퍼 사용 (임계 구역 및 상호 배제 축소)
 - 작업 단위 : 텍스트 파일 한 줄

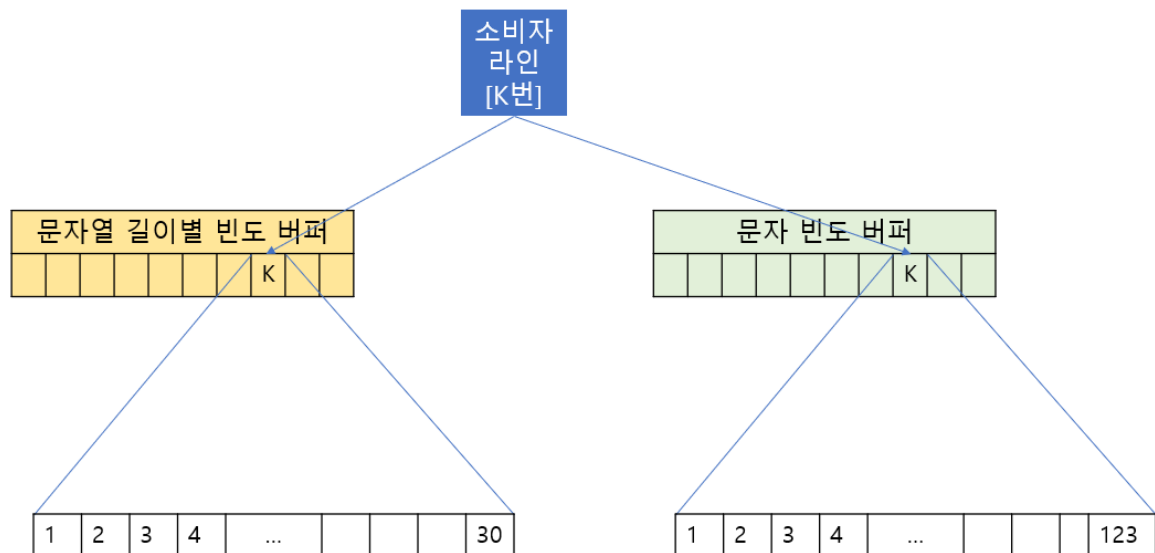
- Ver 4 : 작업단위 증가 + 상호 배제 축소
 - 생산자 스레드 : 생산자 라인 변수 공유(파일로부터 읽어올 줄, 저장할 버퍼번호)
 - 소비자 스레드 :
 - ① 소비자 라인 변수 공유(읽어올 버퍼 번호)
 - ② 스레드 별 통계 결과 저장할 독립된 버퍼 사용 (임계 구역 및 상호 배제 축소)

- 작업 단위 : 텍스트 파일 여러 줄(1000줄, 10000줄, 100000줄..)

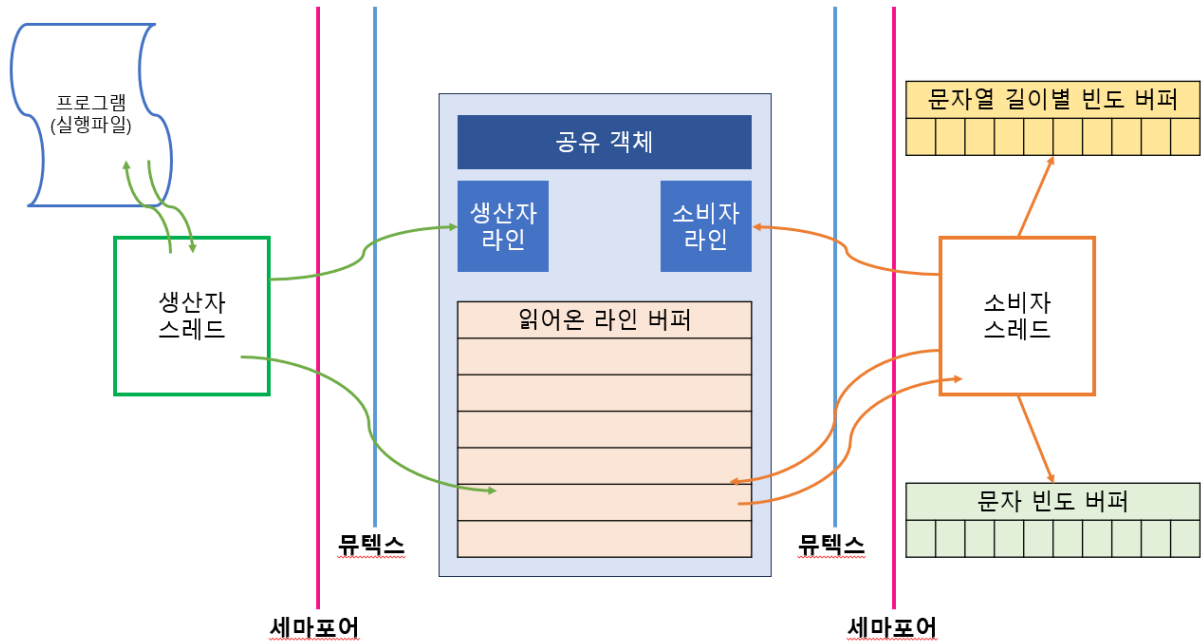


[그림 10] Ver.1 및 Ver.2(작업 단위만 변경) 구조

위의 그림10에서 보듯 Ver.1과 Ver.2에서는 통계 결과 저장 공간을 공유하여 사용한다. 문자열 길이별 빈도 버퍼에는 1부터 30문자열까지의 빈도를 저장하고, 문자 빈도 버퍼에는 A부터 z까지의 빈도가 저장된다.



[그림 11-1] Ver.3, Ver.4에 적용되는 개념 (스레드 별 통계 결과 저장 공간 확장 _ 2차원 배열)



[그림 11-2] Ver.3, Ver4 구조 통계 결과 저장 분리 _ 임계 구역 축소

위의 그림 11-1 에서 보이는 Ver.3 와 Ver.4 에 적용되는 개념은 다음과 같다. 이전에는 빈도 버퍼에 빈도 자체가 저장되는 형태의 1 차원 배열이었던 것과 달리 각 스레드당 통계 공간이 할당되도록 2 차원 배열의 형태로 변경하였다. 따라서 통계 부분은 공유 자원을 사용하지 않으므로 임계구역에 포함하지 않게 되었다. 기존에는 각 버퍼들이 모두 스레드간 공유하는 자원으로써 임계 구역으로 설정되어 컨슈머 스레드에서의 통계 과정도 임계 구역에 포함되었고 그로 인해 대기 시간이 길어진다는 단점이 있었다. 이러한 단점을 통계 과정을 임계 구역에서 제외하는 방법으로 보완하였다.

Ver.1 구현

이번에 설계한 멀티스레딩 프로그램들의 가장 기본적인 구조가 되는 버전으로 동작 메커니즘은 다음과 같다.

[메커니즘]

```
9  ▾ typedef struct sharing_obj{
10      int product;
11      int consume;
12      FILE* rfile;
13
14      int dashboard1[30]; //문자열 길이
15      int dashboard2[123]; // ASCII : A = 65로 시작, z = 122로 마지막
16      char* lines[100]; // 버퍼 개수(읽어온 라인 저장 포인터 배열)
17
18      sem_t full;
19      sem_t empty;
20      pthread_mutex_t lock;
21 } so;
```

[그림 12] 공유 객체 _ product와 consume는 각각 라인 넘버 저장 변수(공유자원)

```
void* producer(void* sharing_obj){
    so* sp = sharing_obj;
    ssize_t contents = 0; //EOF를 감지 시 -1 저장
    char* line = NULL;
    size_t len = 0;
    int count = 0;
    int* ret = malloc(sizeof(int));

    while(1){
        sem_wait(&sp->empty);
        pthread_mutex_lock(&sp->lock);

        contents = getdelim(&line, &len, '\n', sp->rfile);
        if(contents == -1){
            sp->lines[sp->product] = NULL;
            pthread_mutex_unlock(&sp->lock);
            sem_post(&sp->full);
            break;
        }

        sp->lines[sp->product] = strdup(line);
        sp->product = (sp->product + 1)%100;
        count ++;

        pthread_mutex_unlock(&sp->lock);
        sem_post(&sp->full);
    }
    free(line);
    *ret = count;
    pthread_exit(ret);
}
```

페이지 13 / 30

[그림 13] 프로듀서 함수

```

149 void* consumer(void* sharing_obj){
150     so* sp = sharing_obj;
151     char* line = NULL;
152     int count = 0;
153     int* ret = malloc(sizeof(int));
154
155     while(1){
156         sem_wait(&sp->full);
157         pthread_mutex_lock(&sp->lock);
158
159         line = sp->lines[sp->consume];
160         if(line == NULL){
161             pthread_mutex_unlock(&sp->lock);
162             sem_post(&sp->empty);
163             sem_post(&sp->full);
164             break;
165         }
166
167         sp->consume = (sp->consume + 1)%100;
168         count++;
169
170         // 문자열 처리를 위한 복사본 생성
171         char* cptr = strdup(line);
172         char* brka = NULL;
173         char* substr = NULL;
174         const char* sep = "{}()[];\\\" \\n\\t^";
175
176         // 토큰화하여 각 단어 처리
177         for(substr = strtok_r(cptr, sep, &brka); substr; substr = strtok_r(NULL, sep, &brka)) { ...
178     }
179
180     free(cptr); // 복사본 메모리 해제
181     free(line); // 처리 완료된 라인 메모리 해제
182
183     pthread_mutex_unlock(&sp->lock);
184     sem_post(&sp->empty);
185 }
186
187 *ret = count;
188 pthread_exit(ret);
189 }

```

[그림 14] 컨슈머 함수 _ 통계부분도 임계 구역 내 존재 (관련 내용 뒤에 다룰 예정)

```

sem_init(&sp->full,0,0);
sem_init(&sp->empty,0,100);
pthread_mutex_init(&sp->lock,NULL);

```

세마포어, 뮤텍스 초기화

뮤텍스의 쓰임 :

- 라인 넘버를 저장하는 공유 자원인 product 와 consume 를 수정하기 위해 사용

세마포어의 쓰임:

- 라인 버퍼에 남는 공간 현황을 나타내는 full 과 empty 를 수정하기 위해 사용

Ver.1의 실행 결과

Ver.1 Prod 1 : Cons 1

```
main: consumer_0 joined with 21262918
main: producer_0 joined with 21262918

*** print out distributions ***
#ch freq
[ 1]: 12409260 *****
[ 2]: 11541548 *****
[ 3]: 9170747 *****
[ 4]: 10942651 *****
[ 5]: 5000577 *****
[ 6]: 9907805 *****
[ 7]: 4530514 *****
[ 8]: 3091007 *****
[ 9]: 2073134 *****
[10]: 3213947 *****
[11]: 1502238 *****
[12]: 1195846 *****
[13]: 1011957 *****
[14]: 821582 *****
[15]: 752459 *****
[16]: 694808 *****
[17]: 476603 *****
[18]: 457023 *****
[19]: 353100 *****
[20]: 352511 *****
[21]: 266901 *****
[22]: 216479 *****
[23]: 192081 *****
[24]: 160833 *****
[25]: 134201 *****
[26]: 104878 *****
[27]: 91846 *****
[28]: 82436 *****
[29]: 64247 *****
[30]: 582876 *****

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
26327129 8637394 19023049 16795371 40396112
Elapsed time: 23.45 seconds
```

Ver.1 Prod 50 : Cons 50

```
[ 27]: 91846
[ 28]: 82436
[ 29]: 64247
[ 30]: 582876

A B C D E
26327129 8637394 19023049 16795371 40396112
Elapsed time: 141.98 seconds
```

Ver.1 Prod 100 : Cons 100

```
[ 27]: 91846
[ 28]: 82436
[ 29]: 64247
[ 30]: 582876

A B C D E
26327129 8637394 19023049 16795371 40396112
Elapsed time: 152.46 seconds
```

실행 결과: 스레드의 수가 늘어날수록 실행시간이 늘어났다. 1:1의 경우 23초, 50:50의 경우 142초, 100:100의 경우 152초로 거의 7배 가까이 차이가 남을 확인할 수 있었다. 효율적으로 작업을 수행하기 위해 멀티스레딩을 구현하였음에도 오히려 비효율적인 결과가 나온 것이다. 그렇다면 과연

어디에서 오버헤드가 발생하였을까? 오버헤드가 발생할 수 있는 부분들을 살펴보고 분석해보았고 결론적으로 두가지 문제점을 찾을 수 있었다.

[문제 1: 긴 임계 구역, 공유 자원 대상으로 한 병목 현상으로 인한 오버헤드]

스레드를 늘릴수록 오히려 실행시간이 길어진다는 것은 먼저 간단하게 생각했을 때 병목 현상을 떠올릴 수 있다. 그래서 가장 먼저 병목 현상이 일어날 수 있는 부분을 찾아보았다. 그 결과,

- 프로듀서 스레드에서는
 - “생산자 라인 넘버 변수”
- 컨슈머 스레드에서는
 - “소비자 라인 넘버 변수”
 - 통계 결과 저장 버퍼1(문자열길이별 빈도)
 - 통계 결과 저장 버퍼2(문자별 빈도)

에서 병목현상이 발생하고 있음을 확인하였다. 필자는 라인 넘버 변수의 경우 각 생산자와 소비자가 자신의 작업 내용을 식별할 수 있도록 하는 기능을 하기 때문에 여러 스레드가 필수적으로 공유해야 하는 자원으로 생각하였다. 하지만 컨슈머 스레드에서의 통계 결과를 저장하는 버퍼 2 종류의 경우 각 컨슈머 별로 각자 통계를 내고 모든 작업이 끝난 후 각자의 결과를 종합하면 되기 때문에 굳이 공유할 필요가 없다고 생각하여 컨슈머 스레드 별로 개별적인 저장공간을 주고 이 부분을 동기화 영역에서 제외하기로 결정하였다. 프로듀서 스레드와 달리 컨슈머 스레드의 경우 읽어온 정보를 가지고 통계를 내는 작업도 포함하기 때문에 이 작업을 진행하는 시간 또한 다음 스레드가 대기시간에 영향을 준다. 따라서 이 부분을 동기화 영역에서 제외하는 방법은 임계 구역을 축소하는 효과와 동기화 기간을 단축시키는 효과 두 가지를 갖는다고 볼 수 있다.

[해결 방법: 각 스레드 개별적인 자원 할당하여 동기화 영역을 축소 -> 동기화 시간 감소]

[문제 2: 너무 작은 작업 단위로 인한 동기화의 높은 빈도]

한 가지 더 발견한 문제로는 스레드 개수와 상관없이 실행 시간이 너무 많이 걸린다는 점이였다. 물론 23초도 긴 시간은 아니지만 필자는 좀 더 빠른 결과를 원했다. 아무래도 스레드들의 작업 단위가 텍스트 파일에서의 한 줄이다 보니 원본 파일의 라인 수가 많은 경우 라인 수만큼 동기화가 이루어져야 했다. 과도한 동기화 빈도는 대기시간을 증가시켜 전체적인 오버헤드가 커지게 만든다. 현재 버전은 한 줄을 작업 단위로 두기 때문에 동기화 횟수는 텍스트 파일의 전체 라인 수와 같다. 따라서 작업단위를 1000줄로 설정하는 경우 동기화 횟수는 1/1000로, 10,000줄로 설정하

는 경우 1/10,000로 줄어 들 것으로 판단할 수 있다. 하지만 이 경우 프로그램 자체에서 필요한 메모리의 크기가 증가한다는 단점이 있다. 그러나 작업 대상을 저장할 공간이 1000배 늘어난다고 하더라도 절대적인 크기는 그리 큰 편은 아니므로 효율적인 측면에서 긍정적으로 판단하였다.

개선 방안 정리:

- 1) 스레드의 개수가 늘어날수록 실행 시간이 길어지는 문제:
 - 현재 불필요하게 동기화가 이뤄지는 부분을 줄여서 꼭 필요한 부분에서만 동기화를 진행하는 방법으로 해결한다.
- 2) 실행시간 자체가 너무 오래 걸리는 점
 - 작업 단위를 크게 하여 동기화 빈도를 줄인다.

Ver.2, Ver.3에 추가되는 사항 구현

```
#define BATCH_SIZE 1000 //읽어들일 줄 수 설정
#define BUFFER_SIZE 100

typedef struct line_batch { // 기존의 한 줄 단위가 아닌 여러 줄을 읽기위해 새로운 구조체 도입
    char** lines;
    int count; // 실제 배치에 포함된 라인 수
    int is_last; // 마지막 배치 여부
} line_batch;

typedef struct sharing_obj {
    int product;
    int consume;
    FILE* rfile;
    int eof_reached; // EOF 도달 여부//한번에 여러줄을 읽어야 하기 때문

    int dashboard1[30]; // 문자열 길이
    int dashboard2[123]; // ASCII : A = 65로 시작, z = 122로 마지막
    line_batch* batches[BUFFER_SIZE]; // 배치 버퍼

    sem_t full;
    sem_t empty;
    pthread_mutex_t lock;
    pthread_mutex_t file_lock; // 파일 읽기용 뮤텝스
    pthread_mutex_t dashboards_lock; // 대시보드 업데이트용 뮤텝스
} so;
```

[그림 18] Ver.2에 추가되는 기능 구현 _ 작업 단위 증가(여러 줄 읽기)

```

#define NUM_BUFFERS 100

typedef struct sharing_obj {
    int product;          // 다음에 저장할 버퍼 번호
    int consume;          // 소비할 버퍼 번호
    FILE* rfile;

    int dashboard1[ NUM_BUFFERS ][ 30 ]; // 문자열 길이 (스레드별) 최대 스레드 100개이므로 100개 공간 마련
    int dashboard2[ NUM_BUFFERS ][ 123 ]; // ASCII 문자 빈도 (스레드별) 최대 스레드 100개이므로 100개 공간 마련
    char* lines[100];      // 버퍼 배열

    sem_t full;
    sem_t empty;
    pthread_mutex_t lock;
} so;

```

[그림 19] Ver.3에 추가되는 기능 구현 _ 2차원 배열을 이용한 스레드 별 통계 저장 공간 마련(공유하는 자원들을 축소)

```

while(1) {
    sem_wait(&sp->full);
    pthread_mutex_lock(&sp->lock);

    // 버퍼에서 데이터 가져오기
    line = sp->lines[sp->consume];
    if(line == NULL) {
        // EOF 표시된 버퍼 발견
        pthread_mutex_unlock(&sp->lock);
        sem_post(&sp->empty);
        sem_post(&sp->full);
        break;
    }

    sp->consume = (sp->consume + 1) % NUM_BUFFERS;
    count++;

    pthread_mutex_unlock(&sp->lock);
    sem_post(&sp->empty);

    // 문자열 처리를 위한 복사본 생성
    char* cptr = strdup(line);
    char* brka = NULL;
    char* substr = NULL;
    const char* sep = "{ } ( ) [ ] , ; \n \t ^";

    // 토큰화하여 각 단어 처리 - thread_id에 해당하는 배열에 저장
    for(substr = strtok_r(cptr, sep, &brka); substr; substr = strtok_r(NULL, sep, &brka)) {
        size_t length = strlen(substr);
        if(length >= 30) length = 30;

        sp->dashboard1[thread_id][length-1]++;

        for(int i = 0; substr[i] != '\0'; i++) {

```

[그림 20] Ver.3의 컨슈머 함수에서 임계 구역이 축소된 모습 _ 통계 파트 분리

Ver.2 & Ver.3의 실행 결과

다음은 위에서 다룬 개선 방안을 각각 하나씩 적용한 업그레이드 버전의 실행 결과이다.

Ver.2: 기본형에서 작업단위를 늘린 버전

Ver.3: 기본형에서 동기화 방식을 최적화한 버전

결과 예상:

Ver.2의 경우 기본형과 동일하게 스레드가 늘어날수록 시간이 더 걸리나 그 절대적인 양 자체는 줄어든 것으로 예상하였다.

Ver.3의 경우 기본형과 달리 스레드가 늘어날수록 시간이 줄어든 것으로 예상하였다.

다음은 그 결과이다.

[Ver.2의 결과]

```
*** print out distributions ***
#ch  freq
[ 1]: 12499260 *****
[ 2]: 11541548 *****
[ 3]: 9170747 *****
[ 4]: 10342651 *****
[ 5]: 5000577 *****
[ 6]: 9907805 *****
[ 7]: 4530514 *****
[ 8]: 3891007 *****
[ 9]: 2673334 *****
[10]: 3213947 *****
[11]: 1502238 *****
[12]: 1195846 *****
[13]: 1011957 *****
[14]: 821582 *****
[15]: 752459 *****
[16]: 604860 *****
[17]: 476603 *****
[18]: 457823 *****
[19]: 353180 *****
[20]: 352511 *****
[21]: 266901 *****
[22]: 216479 *****
[23]: 192081 *****
[24]: 160833 *****
[25]: 134201 *****
[26]: 104878 *****
[27]: 91846 *****
[28]: 82436 *****
[29]: 64247 *****
[30]: 582876 *****

A B C D
26327129 8637394 19023049 16795371 403
Elapsed time: 10.88 seconds
```

1:1, 1,000줄 단위

```
*** print out distributions ***
#ch  freq
[ 1]: 12499260 *****
[ 2]: 11541548 *****
[ 3]: 9170747 *****
[ 4]: 10342651 *****
[ 5]: 5000577 *****
[ 6]: 9907805 *****
[ 7]: 4530514 *****
[ 8]: 3891007 *****
[ 9]: 2673334 *****
[10]: 3213947 *****
[11]: 1502238 *****
[12]: 1195846 *****
[13]: 1011957 *****
[14]: 821582 *****
[15]: 752459 *****
[16]: 604860 *****
[17]: 476603 *****
[18]: 457823 *****
[19]: 353180 *****
[20]: 352511 *****
[21]: 266901 *****
[22]: 216479 *****
[23]: 192081 *****
[24]: 160833 *****
[25]: 134201 *****
[26]: 104878 *****
[27]: 91846 *****
[28]: 82436 *****
[29]: 64247 *****
[30]: 582876 *****

A B C D
26327129 8637394 19023049 16795371 403
Elapsed time: 33.21 seconds
```

50:50, 1,000줄 단위

```
*** print out distributions ***
#ch  freq
[ 1]: 12499260 *****
[ 2]: 11541548 *****
[ 3]: 9170747 *****
[ 4]: 10342651 *****
[ 5]: 5000577 *****
[ 6]: 9907805 *****
[ 7]: 4530514 *****
[ 8]: 3891007 *****
[ 9]: 2673334 *****
[10]: 3213947 *****
[11]: 1502238 *****
[12]: 1195846 *****
[13]: 1011957 *****
[14]: 821582 *****
[15]: 752459 *****
[16]: 604860 *****
[17]: 476603 *****
[18]: 457823 *****
[19]: 353180 *****
[20]: 352511 *****
[21]: 266901 *****
[22]: 216479 *****
[23]: 192081 *****
[24]: 160833 *****
[25]: 134201 *****
[26]: 104878 *****
[27]: 91846 *****
[28]: 82436 *****
[29]: 64247 *****
[30]: 582876 *****

A B C D
26327129 8637394 19023049 16795371 403
Elapsed time: 40.72 seconds
```

100:100, 1,000줄 단위

```
[ 27]: 91846
[ 28]: 82436
[ 29]: 64247
[ 30]: 582876

A B C
26327129 8637394 19023049 1
Elapsed time: 8.86 seconds
```

1:1, 10,000줄 단위

```
[ 27]: 91846
[ 28]: 82436
[ 29]: 64247
[ 30]: 582876

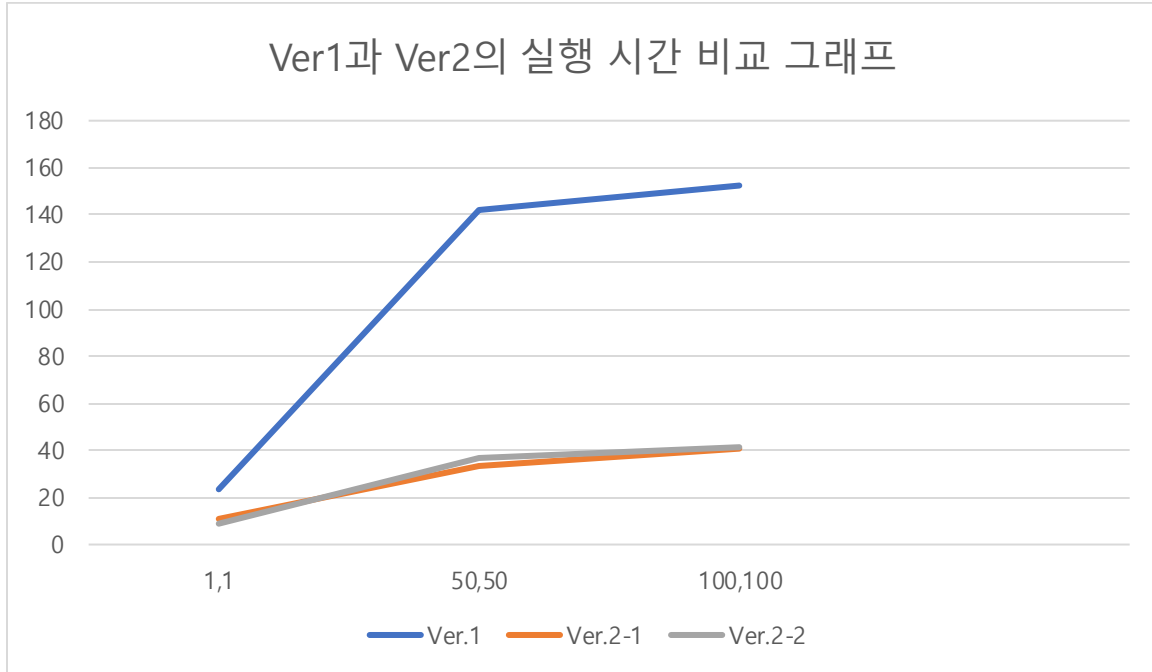
A B C
26327129 8637394 19023049 10
Elapsed time: 36.92 seconds
```

50:50, 10,000줄 단위

```
[ 27]: 91846
[ 28]: 82436
[ 29]: 64247
[ 30]: 582876

A B C
26327129 8637394 19023049 16
Elapsed time: 41.34 seconds
```

100:100, 10,000줄단위



[실행 시간 비교 그래프] Ver.2-1은 1,000줄 단위, Ver.2-2는 10,000줄 단위를 의미

[Ver.2 결과 분석]

이전에 예측했던 대로 단위 작업량을 늘리니 동기화 횟수가 줄어 스레드들이 대기하는 시간이 감소하여 실행 시간이 대폭 감소한 것을 확인할 수 있다. 그러나 스레드가 늘어날수록 오버헤드가 증가하여 실행시간이 증가하는 우상향의 기조는 여전히 유지하고 있는 것을 확인할 수 있다. 그런데 1,000 줄의 작업단위와 10,000 줄의 작업단위의 실행에서의 효율이 거의 비슷한 것으로 나타나고 있다. 이는 작업량이 너무 많아지는 경우에 대해서 그만큼 처리해야 하는 양이 많아지므로 그에 따른 비효율이 발생하여 두 결과가 비슷한 것으로 추측된다.

[Ver.3의 결과]

다음은 동기화 방식을 최적화한 버전 3의 결과이다.

```
*** print out distributions ***
#ch freq
[ 1]: 12499260 *****
[ 2]: 11541548 *****
[ 3]: 9170747 *****
[ 4]: 10342651 *****
[ 5]: 5000577 *****
[ 6]: 9907805 *****
[ 7]: 4530514 *****
[ 8]: 3891007 *****
[ 9]: 2673334 *****
[10]: 3213947 *****
[11]: 1502238 *****
[12]: 1195846 *****
[13]: 1011957 *****
[14]: 821582 *****
[15]: 752459 *****
[16]: 604860 *****
[17]: 476603 *****
[18]: 457823 *****
[19]: 353180 *****
[20]: 352511 *****
[21]: 266901 *****
[22]: 216479 *****
[23]: 192081 *****
[24]: 160833 *****
[25]: 134201 *****
[26]: 104878 *****
[27]: 91846 *****
[28]: 82436 *****
[29]: 64247 *****
[30]: 582876 *****

A B C D
26327129 8637394 19023049 16795371 4039
Elapsed time: 37.20 seconds
```

```
*** print out distributions ***
#ch freq
[ 1]: 12499260 *****
[ 2]: 11541548 *****
[ 3]: 9170747 *****
[ 4]: 10342651 *****
[ 5]: 5000577 *****
[ 6]: 9907805 *****
[ 7]: 4530514 *****
[ 8]: 3891007 *****
[ 9]: 2673334 *****
[10]: 3213947 *****
[11]: 1502238 *****
[12]: 1195846 *****
[13]: 1011957 *****
[14]: 821582 *****
[15]: 752459 *****
[16]: 604860 *****
[17]: 476603 *****
[18]: 457823 *****
[19]: 353180 *****
[20]: 352511 *****
[21]: 266901 *****
[22]: 216479 *****
[23]: 192081 *****
[24]: 160833 *****
[25]: 134201 *****
[26]: 104878 *****
[27]: 91846 *****
[28]: 82436 *****
[29]: 64247 *****
[30]: 582876 *****

A B C D
26327129 8637394 19023049 16795371 4039
Elapsed time: 19.35 seconds
```

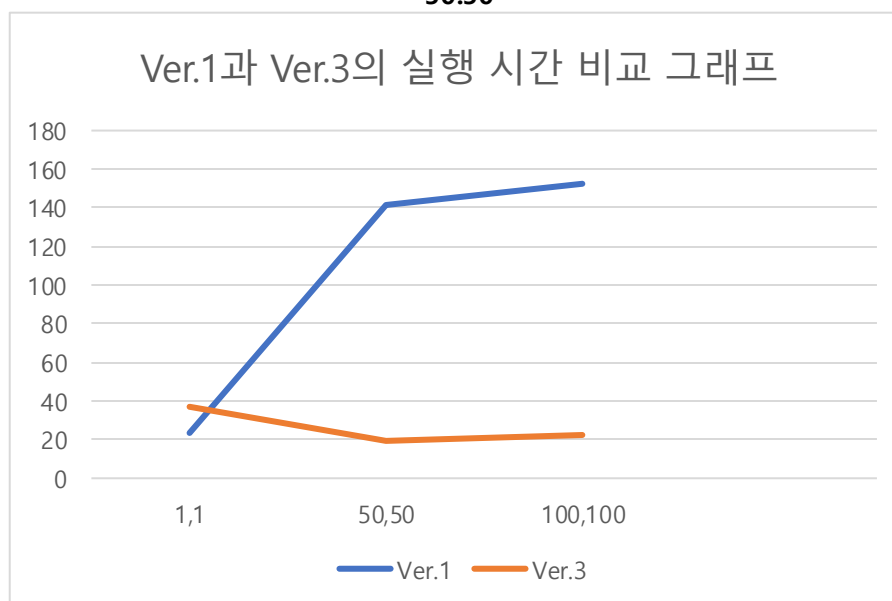
```
*** print out distributions ***
#ch freq
[ 1]: 12499260 *****
[ 2]: 11541548 *****
[ 3]: 9170747 *****
[ 4]: 10342651 *****
[ 5]: 5000577 *****
[ 6]: 9907805 *****
[ 7]: 4530514 *****
[ 8]: 3891007 *****
[ 9]: 2673334 *****
[10]: 3213947 *****
[11]: 1502238 *****
[12]: 1195846 *****
[13]: 1011957 *****
[14]: 821582 *****
[15]: 752459 *****
[16]: 604860 *****
[17]: 476603 *****
[18]: 457823 *****
[19]: 353180 *****
[20]: 352511 *****
[21]: 266901 *****
[22]: 216479 *****
[23]: 192081 *****
[24]: 160833 *****
[25]: 134201 *****
[26]: 104878 *****
[27]: 91846 *****
[28]: 82436 *****
[29]: 64247 *****
[30]: 582876 *****

A B C D
26327129 8637394 19023049 16795371 4039
Elapsed time: 22.25 seconds
```

1:1

50:50

100:100



[Ver.3 결과 분석]

Ver.3의 경우 이전에 예측했던 대로 스레드가 늘어날수록 실행 시간이 늘어나는 우상향 그래프를 그리는 Ver.1과는 달리 스레드가 늘어날수록 실행시간이 줄어드는 우하향 그래프를 그리고 있다. 1:1에서는 Ver.1이 Ver.3보다 빠르기는 하나 그 이후로 50:50, 100:100에서는 Ver.3이 월등히 빠른 성능을 보여주고 있다.

[Ver.4의 결과]

다음은 작업량 증가와 동기화 최적화한 Ver.4의 실행 결과이다.

1) 작업량 조작

```
*** print out distributions ***
#ch  freq
[ 1]: 12499260 *****
[ 2]: 11541548 *****
[ 3]: 9170747 *****
[ 4]: 10342651 *****
[ 5]: 5000577 *****
[ 6]: 9907805 *****
[ 7]: 4530514 *****
[ 8]: 3891007 *****
[ 9]: 2673334 *****
[10]: 3213947 *****
[11]: 1502238 *****
[12]: 1195846 *****
[13]: 1011957 *****
[14]: 821582 *****
[15]: 752459 *****
[16]: 604860 *****
[17]: 476603 *****
[18]: 457823 *****
[19]: 353180 *****
[20]: 352511 *****
[21]: 266901 *****
[22]: 216479 *****
[23]: 192081 *****
[24]: 160833 *****
[25]: 134201 *****
[26]: 104878 *****
[27]: 91846 *****
[28]: 82436 *****
[29]: 64247 *****
[30]: 582876 *****

A      B      C      D
26327129 8637394 19023049 16795371 4039
Elapsed time: 2.55 seconds
```

결과 1) 100, 100, 10,000줄

```
*** print out distributions ***
#ch  freq
[ 1]: 12499260 *****
[ 2]: 11541548 *****
[ 3]: 9170747 *****
[ 4]: 10342651 *****
[ 5]: 5000577 *****
[ 6]: 9907805 *****
[ 7]: 4530514 *****
[ 8]: 3891007 *****
[ 9]: 2673334 *****
[10]: 3213947 *****
[11]: 1502238 *****
[12]: 1195846 *****
[13]: 1011957 *****
[14]: 821582 *****
[15]: 752459 *****
[16]: 604860 *****
[17]: 476603 *****
[18]: 457823 *****
[19]: 353180 *****
[20]: 352511 *****
[21]: 266901 *****
[22]: 216479 *****
[23]: 192081 *****
[24]: 160833 *****
[25]: 134201 *****
[26]: 104878 *****
[27]: 91846 *****
[28]: 82436 *****
[29]: 64247 *****
[30]: 582876 *****

A      B      C      D
26327129 8637394 19023049 16795371 4039
Elapsed time: 3.36 seconds
```

결과 2) 100, 100, 100,000줄

```

*** print out distributions ***
#ch  freq
[ 1]: 12499260 *****
[ 2]: 11541548 *****
[ 3]: 9170747 *****
[ 4]: 10342651 *****
[ 5]: 5000577 *****
[ 6]: 9907805 *****
[ 7]: 4530514 *****
[ 8]: 3891007 *****
[ 9]: 2673334 *****
[10]: 3213947 *****
[11]: 1502238 *****
[12]: 1195846 *****
[13]: 1011957 *****
[14]: 821582 *****
[15]: 752459 *****
[16]: 604860 *****
[17]: 476603 *****
[18]: 457823 *****
[19]: 353180 *****
[20]: 352511 *****
[21]: 266901 *****
[22]: 216479 *****
[23]: 192081 *****
[24]: 160833 *****
[25]: 134201 *****
[26]: 104878 *****
[27]: 91846 *****
[28]: 82436 *****
[29]: 64247 *****
[30]: 582876 *****

A B C D
26327129 8637394 19023049 16795371 46
Elapsed time: 6.29 seconds

```

```

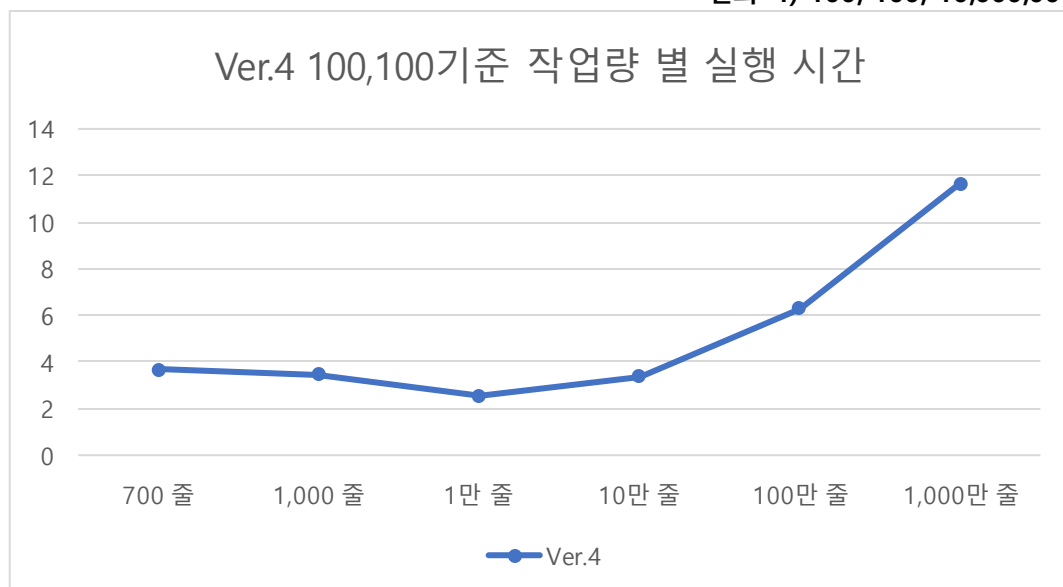
*** print out distributions ***
#ch  freq
[ 1]: 12499260 *****
[ 2]: 11541548 *****
[ 3]: 9170747 *****
[ 4]: 10342651 *****
[ 5]: 5000577 *****
[ 6]: 9907805 *****
[ 7]: 4530514 *****
[ 8]: 3891007 *****
[ 9]: 2673334 *****
[10]: 3213947 *****
[11]: 1502238 *****
[12]: 1195846 *****
[13]: 1011957 *****
[14]: 821582 *****
[15]: 752459 *****
[16]: 604860 *****
[17]: 476603 *****
[18]: 457823 *****
[19]: 353180 *****
[20]: 352511 *****
[21]: 266901 *****
[22]: 216479 *****
[23]: 192081 *****
[24]: 160833 *****
[25]: 134201 *****
[26]: 104878 *****
[27]: 91846 *****
[28]: 82436 *****
[29]: 64247 *****
[30]: 582876 *****

A B C D
26327129 8637394 19023049 16795371 46
Elapsed time: 11.67 seconds

```

결과 3) 100, 100, 1,000,000줄

결과 4) 100, 100, 10,000,000줄



위의 4가지 결과는 최종 버전인 Ver.4의 결과로, 결과1)의 경우 1만줄의 작업 단위와 100:100의 스레드 개수를 사용하여 2.55초를 달성하였고, 결과2)의 경우 십만줄의 작업단위와 100:100의 스레드 개수를 사용하여 3.36초를 달성하였다.

```
main: producer_0 joined with 210000
main: producer_1 joined with 210000
main: producer_2 joined with 220000
main: producer_3 joined with 180000
main: producer_4 joined with 240000
main: producer_5 joined with 220000
main: producer_6 joined with 210000
main: producer_7 joined with 170000
main: producer_8 joined with 190000
main: producer_9 joined with 200000
main: producer_10 joined with 210000
main: producer_11 joined with 240000
main: producer_12 joined with 180000
main: producer_13 joined with 190000
main: producer_14 joined with 222918
main: producer_15 joined with 200000
main: producer_16 joined with 220000
main: producer_17 joined with 180000
main: producer_18 joined with 200000
main: producer_19 joined with 200000
main: producer_20 joined with 180000
main: producer_21 joined with 200000
main: producer_22 joined with 200000
main: producer_23 joined with 240000
main: producer_24 joined with 210000
main: producer_25 joined with 240000
main: producer_26 joined with 200000
main: producer_27 joined with 190000
main: producer_28 joined with 240000
main: producer_29 joined with 230000
main: producer_30 joined with 190000
main: producer_31 joined with 230000
main: producer_32 joined with 210000
```

```
main: consumer_0 joined with 230000
main: consumer_1 joined with 180000
main: consumer_2 joined with 230000
main: consumer_3 joined with 240000
main: consumer_4 joined with 230000
main: consumer_5 joined with 200000
main: consumer_6 joined with 220000
main: consumer_7 joined with 210000
main: consumer_8 joined with 200000
main: consumer_9 joined with 200000
main: consumer_10 joined with 260000
main: consumer_11 joined with 210000
main: consumer_12 joined with 250000
main: consumer_13 joined with 230000
main: consumer_14 joined with 250000
main: consumer_15 joined with 260000
main: consumer_16 joined with 200000
main: consumer_17 joined with 230000
main: consumer_18 joined with 230000
main: consumer_19 joined with 210000
main: consumer_20 joined with 220000
main: consumer_21 joined with 220000
main: consumer_22 joined with 220000
main: consumer_23 joined with 210000
main: consumer_24 joined with 190000
main: consumer_25 joined with 190000
main: consumer_26 joined with 200000
main: consumer_27 joined with 220000
main: consumer_28 joined with 210000
main: consumer_29 joined with 230000
main: consumer_30 joined with 180000
main: consumer_31 joined with 230000
main: consumer_32 joined with 230000
```

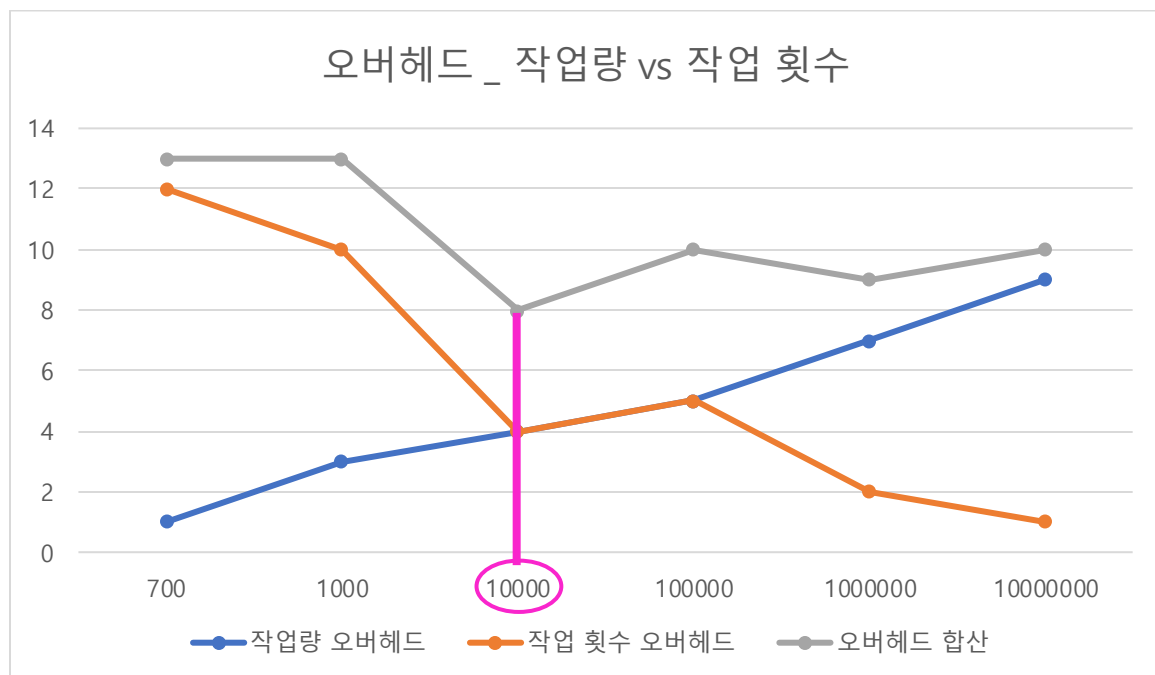
Ver.4 100,100 1만줄 결과 _ 스레드 당 평균 21만개의 라인을 처리한 모습

1 만줄의 작업량의 실행 속도가 가장 빨랐다는 사실로 무엇을 알 수 있을까? 우선 스레드가 1 개일 때의 실행결과를 통해 FreeBSD9-orig.tar 파일의 총 라인 수는 약 2 천 1 백만 줄임을 알 수 있었다.

```
main continuing
main: consumer_0 joined with 21262918
main: producer_0 joined with 21262918
```

FreeBSD9-orig.tar파일의 총 라인 수

총 라인 수가 2 천 1 백만 줄이므로 1 만줄씩 작업을 수행한다고 하면 총 2 천 1 백번 진행하게 되고, 100 개의 프로듀서 스레드가 분업한다고 하면 각 스레드 당 평균 21 번 작업을 수행하고 종료된다고 추측할 수 있다. 또한, 10 만줄씩 작업을 수행한다고 하면 총 210 번 진행하게 되고, 100 개의 프로듀서 스레드가 분업할 시 스레드 당 평균 2.1 번 작업을 수행한다. 따라서 이것으로 알 수 있는 점은 스레드 실행횟수(작업 실행 횟수, 동기화 횟수)가 2 천 1 백번에서 210 번으로 줄어들 때 얻어지는 이득보다 각 스레드 내부에서 일어나는 작업량이 1 만줄에서 10 만줄로 늘어날 때 얻어지는 오버헤드가 더 크다는 것이다. 정리하자면 빠르게 2 천 1 백번 실행하는 것이 느리게 210 번 실행하는 것보다 효율적인 결과를 낳았다고 할 수 있다. 하지만 그와 반대로 1000 줄, 700 줄의 작업량에서는 1 만줄의 작업량에서의 실행 시간보다 긴 시간이 걸렸는데, 이 경우에는 1 만줄보다 적은 작업량으로 단위작업속도는 빨랐지만 그 횟수가 늘어났기 때문이라고 해석할 수 있다. 결론적으로 작업량으로 인한 오버헤드와 작업횟수로 인한 오버헤드의 합이 1 만 줄에서 제일 적었다고 이해할 수 있다.



위의 그래프는 임의로 만든 그래프로, 작업량과 작업 횟수로 인한 오버헤드의 양상을 추측하여 나타내었다. 오버헤드의 합산이 10000 줄의 작업량에서 제일 적게 나왔다는 것을 표현하고 있다.

```

main continuing
main: consumer_0 joined with 0
main: consumer_1 joined with 0
main: consumer_2 joined with 10000000
main: consumer_3 joined with 10000000
main: consumer_4 joined with 0
main: consumer_5 joined with 0
main: consumer_6 joined with 1262918
main: consumer_7 joined with 0
main: consumer_8 joined with 0

```

1천만 줄 케이스

```

main: consumer_99 joined with 0
main: producer_0 joined with 10000000
main: producer_1 joined with 1262918
main: producer_2 joined with 10000000
main: producer_3 joined with 0
main: producer_4 joined with 0
main: producer_5 joined with 0
main: producer_6 joined with 0

```

1천만 줄 케이스

```

main: consumer_21 joined with 1000000
main: consumer_22 joined with 0
main: consumer_23 joined with 1000000
main: consumer_24 joined with 1000000
main: consumer_25 joined with 1000000
main: consumer_26 joined with 1000000
main: consumer_27 joined with 1000000
main: consumer_28 joined with 1000000
main: consumer_29 joined with 1000000
main: consumer_30 joined with 1000000
main: consumer_31 joined with 1000000
main: consumer_32 joined with 1000000
main: consumer_33 joined with 1000000
main: consumer_34 joined with 1000000
main: consumer_35 joined with 1000000
main: consumer_36 joined with 1000000
main: consumer_37 joined with 1000000
main: consumer_38 joined with 1000000
main: consumer_39 joined with 1000000
main: consumer_40 joined with 1000000
main: consumer_41 joined with 1000000
main: consumer_42 joined with 0
main: consumer_43 joined with 0
main: consumer_44 joined with 0
main: consumer_45 joined with 0
main: consumer_46 joined with 0
main: consumer_47 joined with 0
main: consumer_48 joined with 0
main: consumer_49 joined with 0
main: consumer_50 joined with 0
main: consumer_51 joined with 262918

```

+)1백만 줄 케이스

추가적으로 위의 좌측 두 자료는 작업량을 1 천만줄로 설정하였을 때 스레드들의 상태를 출력한 것인데 컨슈머, 프로듀서 각각 단 3 개의 스레드 만으로 전 라인을 커버하였음을 알 수 있다. 프로듀서 스레드의 경우 0 번-2 번-1 번(또는 2 번-0 번-1 번) 그리고 그 외의 스레드 순서로 진행되었고 1 번 스레드가 마지막 줄까지 읽어 나머지 스레드들은 생성 후 바로 종료가 되었을 것으로 추측할 수 있다. 이 경우, 라인 넘버 공유 자원에 접근하는 횟수, 즉 임계 구역에 진입하기 위해 상호 배제를 실시하는 횟수가 단 두 차례 밖에 없었을 것으로 판단할 수 있다. (컨슈머 부분에서도 두 차례)따라서 총 걸린 시간인 11.67 초 중 대부분의 시간은 스레드들이 개별적으로 작업을 수행하는 데 걸린 시간과 전체 통계를 합산하는데 걸린 시간일 것으로 추측한다.

4가지 경우를 비교했을 때 1만줄의 케이스가 가장 빨랐기 때문에 앞으로 진행할 스레드 개수 조
작은 1만줄의 작업량의 케이스에 대해 다루겠다.

2) 스레드 개수 조작

```
*** print out distributions ***
#ch  freq
[ 1]: 12499260 *****
[ 2]: 11541548 *****
[ 3]: 9170747 *****
[ 4]: 10342651 *****
[ 5]: 5000577 *****
[ 6]: 9907805 *****
[ 7]: 4530514 *****
[ 8]: 3891007 *****
[ 9]: 2673334 *****
[10]: 3213947 *****
[11]: 1502238 *****
[12]: 1195846 *****
[13]: 1011957 *****
[14]: 821582 *****
[15]: 752459 *****
[16]: 604860 *****
[17]: 476603 *****
[18]: 457823 *****
[19]: 353180 *****
[20]: 352511 *****
[21]: 266901 *****
[22]: 216479 *****
[23]: 192081 *****
[24]: 160833 *****
[25]: 134201 *****
[26]: 104878 *****
[27]: 91846 *****
[28]: 82436 *****
[29]: 64247 *****
[30]: 582876 *****

A B C D
26327129 8637394 19023049 16795371 4
Elapsed time: 3.15 seconds
```

50, 50

```
*** print out distributions ***
#ch  freq
[ 1]: 12499260 *****
[ 2]: 11541548 *****
[ 3]: 9170747 *****
[ 4]: 10342651 *****
[ 5]: 5000577 *****
[ 6]: 9907805 *****
[ 7]: 4530514 *****
[ 8]: 3891007 *****
[ 9]: 2673334 *****
[10]: 3213947 *****
[11]: 1502238 *****
[12]: 1195846 *****
[13]: 1011957 *****
[14]: 821582 *****
[15]: 752459 *****
[16]: 604860 *****
[17]: 476603 *****
[18]: 457823 *****
[19]: 353180 *****
[20]: 352511 *****
[21]: 266901 *****
[22]: 216479 *****
[23]: 192081 *****
[24]: 160833 *****
[25]: 134201 *****
[26]: 104878 *****
[27]: 91846 *****
[28]: 82436 *****
[29]: 64247 *****
[30]: 582876 *****

A B C D
26327129 8637394 19023049 16795371 4
Elapsed time: 3.46 seconds
```

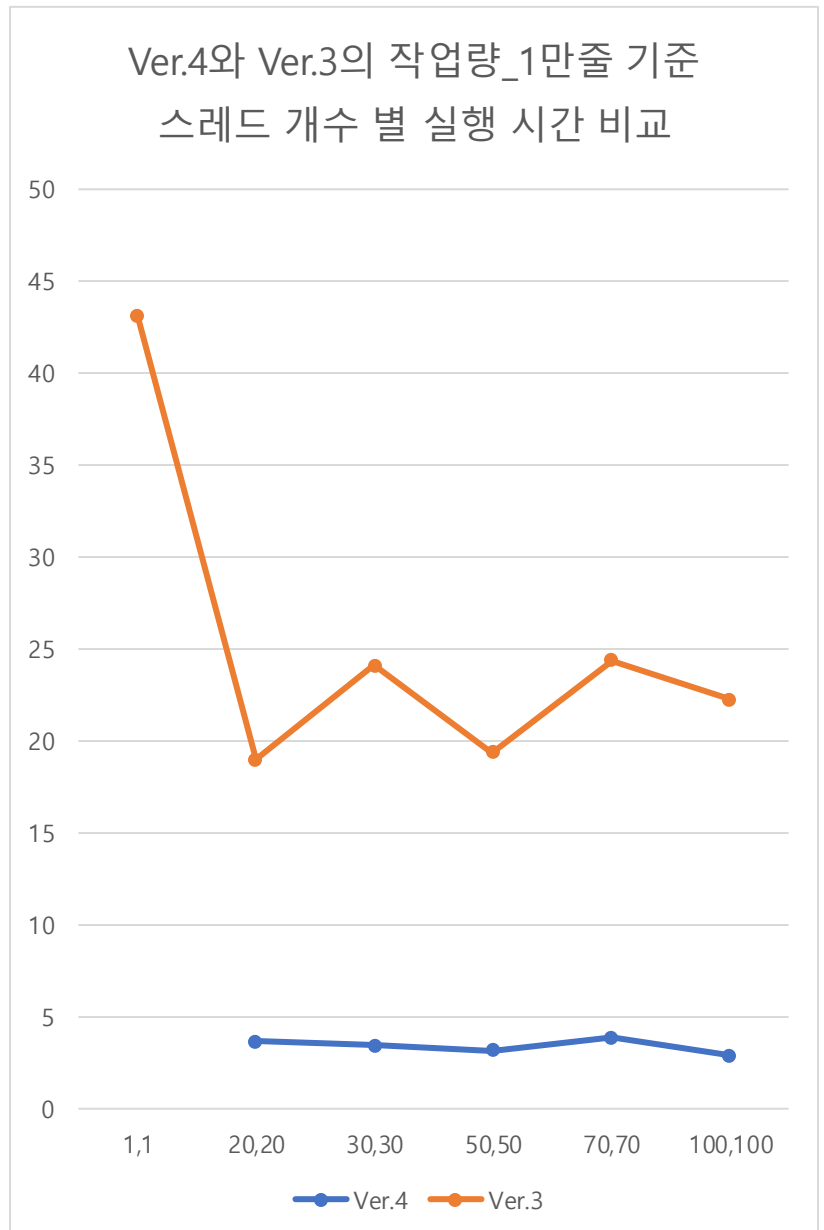
30, 30

```

*** print out distributions ***
#ch  freq
[ 1]: 12499260 *****
[ 2]: 11541548 *****
[ 3]: 9170747 *****
[ 4]: 10342651 *****
[ 5]: 5000577 *****
[ 6]: 9907805 *****
[ 7]: 4530514 *****
[ 8]: 3891007 *****
[ 9]: 2673334 *****
[10]: 3213947 *****
[11]: 1502238 *****
[12]: 1195846 *****
[13]: 1011957 *****
[14]: 821582 *****
[15]: 752459 *****
[16]: 604860 *****
[17]: 476603 *****
[18]: 457823 *****
[19]: 353180 *****
[20]: 352511 *****
[21]: 266901 *****
[22]: 216479 *****
[23]: 192081 *****
[24]: 160833 *****
[25]: 134201 *****
[26]: 104878 *****
[27]: 91846 *****
[28]: 82436 *****
[29]: 64247 *****
[30]: 582876 *****

A B C D
26327129 8637394 19023049 16795371 4
Elapsed time: 3.64 seconds

```



20, 20

위의 그래프는 1 만줄의 작업량을 다루는 Ver.4 와 1 줄의 작업량을 다루는 Ver.3 을 스레드의 개수를 조정해 가면서 측정한 결과값을 그래프로 나타낸 것이다. 그래프에서 알 수 있듯 스레드의 개수가 늘어날수록 병렬처리가 증가하여 대체적으로 실행시간이 줄어들고 있는 것을 확인할 수 있었다. 또한 실행시간의 측면에서 판단할 때, 작업량은 유지한 채 동기화 부분만 최적화 했던 Ver.3 보다 작업량도 늘렸던 Ver.4 에서 훨씬 효율적인 것으로 판단할 수 있었다.

마치며..

지금까지의 과제 중 가장 흥미로웠던 주제여서 재밌게 진행했던 것 같다. 작년 정도에 한번 개인적인 프로젝트를 진행하며 2만개의 데이터를 웹상에서 크롤링해야 했던 적이 있었는데, 그때 데이터 양이 많아 멀티 스레딩 방식으로 접근하려던 시도가 있었다. 그 당시 사실 프로세스도 모르고 스레드도 모르는 무지한 상태에서 주먹구구식으로 멀티 스레딩을 이용하려고 하니 너무나 막막했다. 그러한 기억을 가지고 이번 과제를 진행하다 보니 “아 그때 그래서 다운받아지는 순서가 항상 뒤죽박죽이었구나”, “아 그때 이렇게 하면 시간을 훨씬 더 단축시킬 수 있었겠다” 등의 생각이 들었다. 비록 남아있던 프리데이도 다 사용하고 늦은 제출을 하게 되었지만 그것과는 별개로 통계 출력까지 총 실행시간 2.55초를 달성하였기 때문에 매우 빠른 실행속도의 멀티 스레딩을 구현하게 되어서 기쁘게 생각하는 바이다.

버전 별 실행 방법

- 해당 파일 경로에서 make 입력하여 실행파일 생성
 - Warning은 뜨지만 실행 가능
 - ./실행파일 텍스트파일 프로듀서 컨슈머
 - 작업단위는 ver2에서는 BATCH_SIZE 수정, ver4에서는 BUFFER_SIZE 수정